

3SE2 – Übung 3

Themen: Enumerations, findBy-Queries

In dieser Übung machen wir weiter mit dem Freelancer-Jobportal. Du kannst im bestehenden Projekt weiterarbeiten. Falls du die Musterlösung verwendest, musst du in launch.json die Konfiguration der DB mit deinen Zugangsdaten ergänzen.

Sprint 3 planen

Erstelle im Projekt «Freelancer4u SCRUM Planning» je ein Issue für die folgenden Anforderungen. Plane alle Issues für Sprint 3 und setze sie in den Zustand «Ready». Verwende sinnvolle Labels. Falls nichts passt, erstelle ein neues Label.

Job speichern

POST	/api/job	Fügt einen neuen Job hinzu
Request Body (DTO)	<pre>{ "title": "Review order class", "description": "Check if order class meets all requirements", "jobType": "REVIEW", "earnings": 150.0, "companyId": "6344ef34efc7c92e1ad8412" }</pre>	
Response Status	CREATED	
	INTERNAL_SERVER_ERROR	Speichern fehlgeschlagen
Response Body	DAO	

Alle Jobs auslesen

GET	/api/job	Gibt die Liste aller Jobs zurück
Responses	OK	<pre>[{ "id": "6344ef34efc7c92e1ad83be1", "title": "Review order class", "description": "Check if order class meets ...", "jobType": "REVIEW", "jobState": "NEW", "earnings": 150.0, "companyId": "6344ef34efc7c92e1ad8412", "freelancerId": "" }, {...}]</pre>
Bemerkung	Sind keine Jobs vorhanden, wird OK und ein leeres Array zurückgegeben.	

Job byID lesen

GET	/api/job/{id}	Gibt den Job mit der angegebenen ID zurück
Responses	OK	Im Body DAO der Entität Job
	NOT_FOUND	Company nicht gefunden

Filter für Jobs

Die folgende Tabelle definiert die Anforderungen für die Erweiterung des Job-Endpoints.

Erweiterungen sind gelb markiert

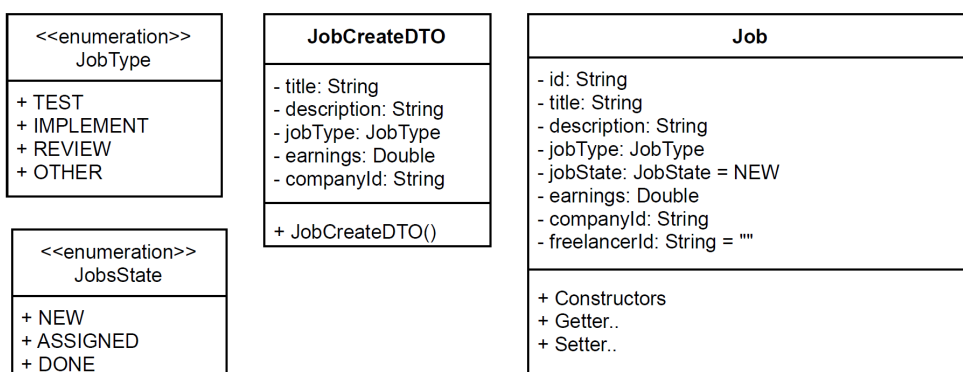
GET	/api/job	Gibt ein Array mit allen Jobs zurück. Falls der Parameter min vorhanden ist, werden nur Jobs zurückgegeben, bei denen der Wert earnings höher ist als der Parameter min. Falls der Parameter type vorhanden, werden nur Jobs zurückgegeben, bei denen der JobType mit dem Parameter type übereinstimmt.
Parameters	min, type	Optionaler Parameter für Minimalwert bei earnings Optionaler Parameter für Filter des JobTypes
Responses	OK	[{"id": "123", "description": "Review main class", "jobType": "REVIEW", "earnings": 150.0, "jobState": "NEW", "companyId": 12, "freelancerId": null }, {...}]
	BAD_REQUEST	Bei ungültigem Wert von min oder type
Beispiele	/api/job	OK, Array mit allen Jobs
	/api/job?min=10	OK, Array mit allen relevanten Jobs
	/api/job?min=zehn	BAD_REQUEST
	/api/job?type=TEST	OK, Array mit allen relevanten Jobs
	/api/job?type=TOAST	BAD_REQUEST
	/api/job?min=10&type=TEST	OK, Array mit allen relevanten Jobs
Bemerkung	Sind keine Jobs vorhanden respektive keine Jobs mit dem angegebenen Minimalwert oder Typ, wird OK und ein leeres Array zurückgegeben.	

Beispiel für das SCRUM-Board

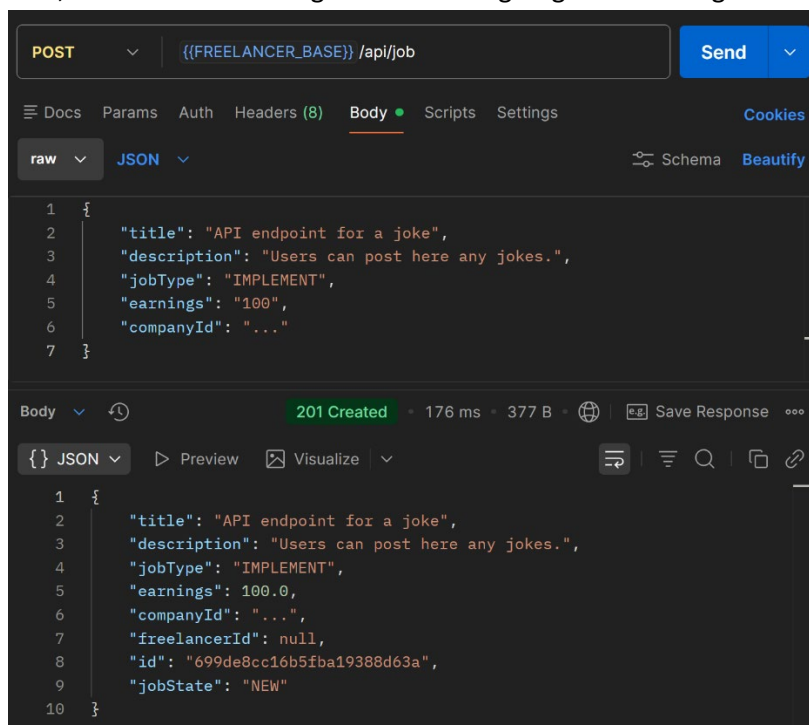
Title	Labels	Status	Iteration
1 Use Case Diagram #1	documentation	Done	Iteration 1
2 API endpoint add company #2	backend enhancement	Done	Iteration 2
3 API endpoint read all companies #3	backend enhancement	Done	Iteration 2
4 API endpoint read company by id #4	backend enhancement	Done	Iteration 2
5 API endpoint to add a job #5	backend enhancement	Ready	Iteration 3
6 API endpoint to get all jobs #6	backend enhancement	Ready	Iteration 3
7 API Endpoint to get job by id #7	backend enhancement	Ready	Iteration 3
8 Extend job endpoint with filter #8	backend enhancement	Ready	Iteration 3

Issue «Job speichern»

- Schiebe das zugehörige Issues im Project-Board in die «In Progress» Spalte.
- Erstelle die benötigten Datenklassen gemäss folgenden Klassendiagrammen. Du benötigst bei Job einen Konstruktor um alle Attribute zu setzen, die über das DTO reinkommen. Verwende einen `@RequiredArgsConstructor` und setze die entsprechenden Attribute auf `@NonNull`. Vergiss nicht die Annotationen `@Id` und `@Document` an der richtigen Stelle einzufügen.



- Füge ein Repository für Jobs hinzu.
- Ergänze das Projekt um einen Controller mit dem benötigten POST Endpoints. Orientiere dich an dem Endpoints für Company. Vergiss nicht das jobRepository einzubinden.
- Teste deine Lösung mit Postman und Compass. Falls du noch keine Collection in Postman hast, erstelle eine. Die folgende Abbildung zeigt einen erfolgreichen POST Request:



- Committe die Änderungen und schliesse das Issue mit einem entsprechenden Kommentar.

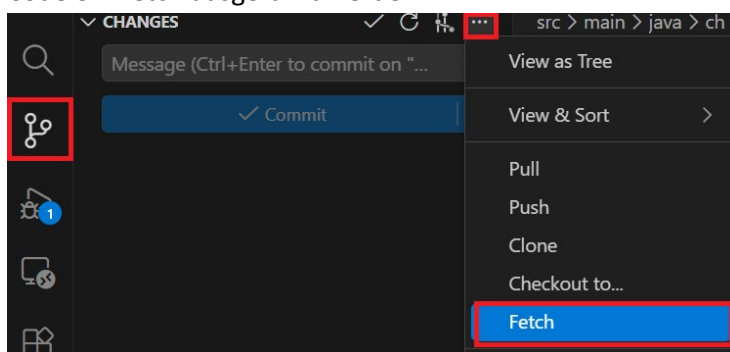
Feature Branch: Issue «Alle Jobs auslesen»

Dieses Issue soll in einem Feature-Branche entwickelt und anschliessend nach Main gemerged werden. Den Branch erstellen wir direkt aus dem Issue.

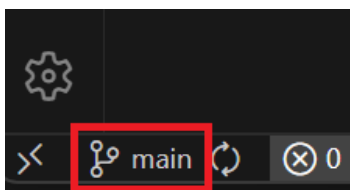
- Schiebe das zugehörige Issues im Project-Board in die «In Progress» Spalte.
- Öffne das Issue und suche das Feld auf der rechten Seite zum Erstellen eines Branches:

The screenshot shows the GitHub 'Create a branch for this issue' dialog. At the top, it says 'Development' with a gear icon. Below it, a red box highlights the link 'Create a branch' with the text 'for this issue or link a pull request.' The dialog itself is titled 'Create a branch for this issue' and has a close button. It contains a 'Branch name' field with the value '6-api-endpoint-to-get-all-jobs'. Below this are two sections: 'Repository destination' with a dropdown showing a repository path, and 'Branch source' with a dropdown showing 'main'. At the bottom, under 'What's next?', there are two radio buttons: 'Checkout locally' (selected) and 'Open branch with GitHub Desktop'. A red box highlights the 'Create branch' button at the bottom right.

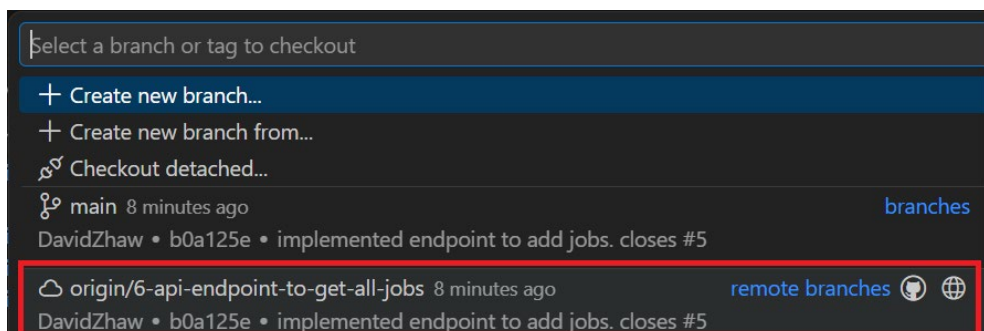
- Der neue Branch ist im Remote-Repo aber noch nicht lokal. Deshalb muss als erstes in VS Code ein Fetch ausgeführt werden.



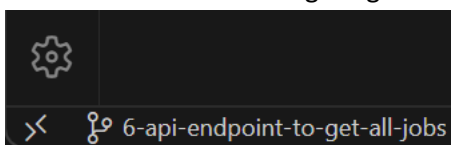
- d) Nun kannst du nach dem klicken auf den Main-Branch



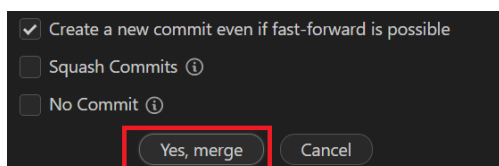
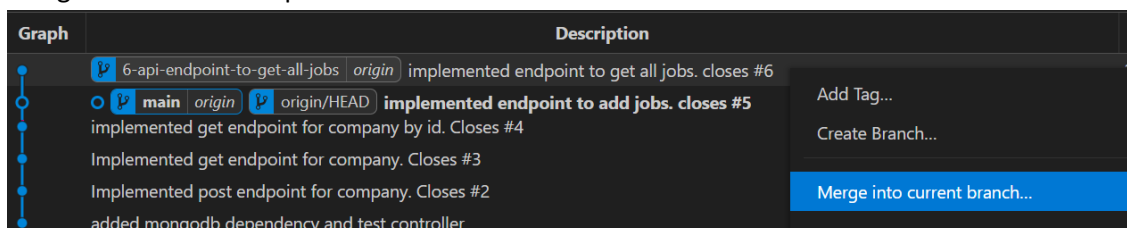
Den neu erstellten Branch auswählen



Dieser sollte nun auch angezeigt werden.



- e) Ergänze den Controller um den benötigten GET Endpoint.
- f) Teste deine Lösung mit Postman und Compass.
- g) Committe die Änderungen und schliesse das Issue mit einem entsprechenden Kommentar. Prüfe im SCRUM Board, ob das Issue geschlossen wurde. Wir stellen fest: es wurde nicht geschlossen!
- h) Merge die Änderungen zurück in den Main-Branch. Wechsle dazu in den Main-Branch und Merge über den GitGraph.

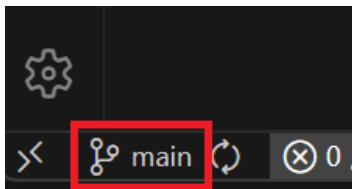


Synchronisiere die Änderungen mit dem Remote-Repo und schau dir nochmals das SCRUM-Board an. Nun sollte das Issue geschlossen sein.

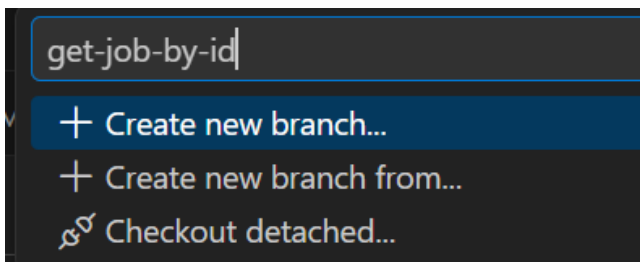
Issue «Job byID lesen»

Auch dieses Issue soll in einem Feature-Branch entwickelt werden. Diesmal machen wir den Branch jedoch im VS Code

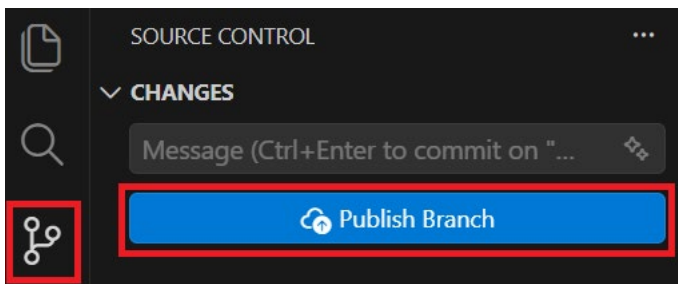
- a) Erstelle einen neuen Branch «get-job-by-id». Klicke auf den Main-Branch



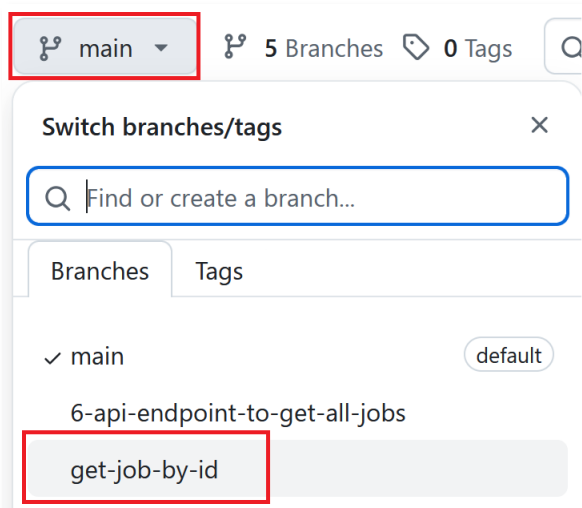
Gibt den Namen ein und wähle «Create new branch..»



Publishe anschliessend den Branch

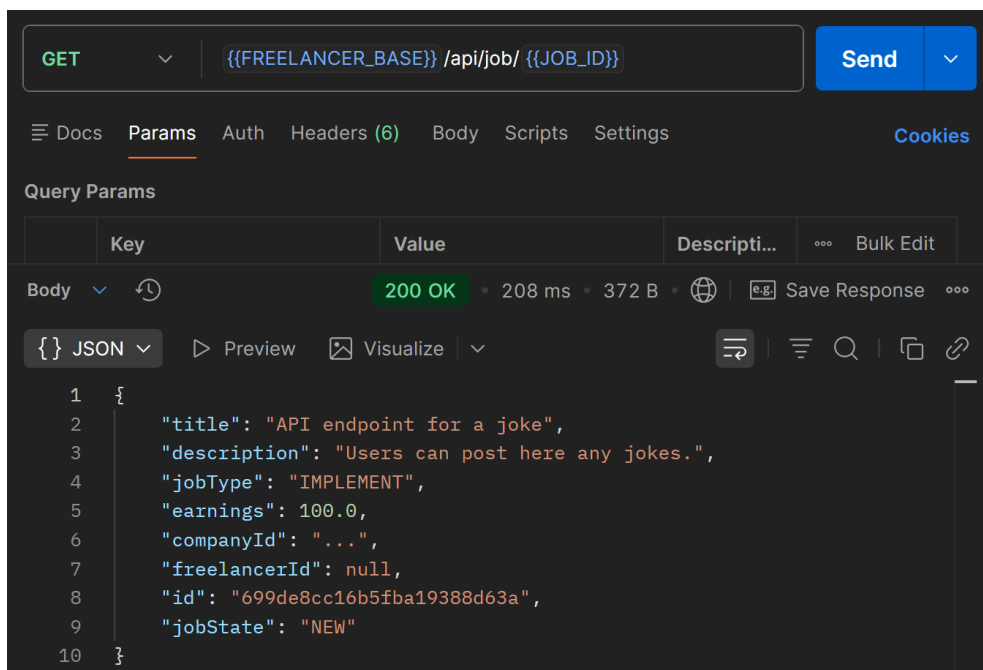


und prüfe in GitHub, ob der neue Branch angezeigt wird.

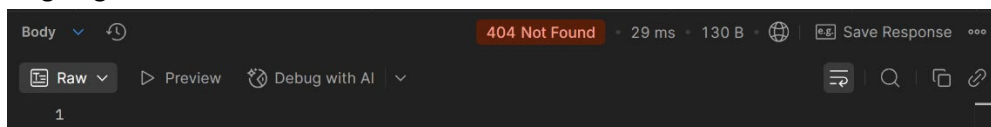


- b) Ergänze den Controller mit dem benötigten Endpoint. Verwende die von MongoRepository geerbte Methode `findById` zum Auslesen der Company. Diese Methode gibt ein `Optional` zurück. Achte darauf, dass du die richtige Library importierst (`java.util.Optional` und nicht die von Apache). Falls die Company gefunden wurde, musst du diese aus dem `Optional` extrahieren und zurückgeben. Falls nicht, musst du eine Response mit dem Status-Code `NOT_FOUND` zurückgeben

- c) Prüfe mit Postman, ob bei einer gültigen ID das Objekt und bei einer ungültigen ID der Status-Code NOT_FOUND zurückkommt. Verwende für die Job-ID eine Variable. Gültige ID:



Ungültige ID



- d) Committe die Änderungen und schliesse das Issue mit einem entsprechenden Kommentar.
- e) Merge die Änderungen nach Main, synchronisiere mit dem Remote-Repo und prüfe, ob das Issue im SCRUM Board geschlossen wurde.

Issue «Filter für Jobs»

Der neue Filter erlaubt es Jobs auszulesen, bei denen der Wert von «earnings» grösser als ein angegebenes Minimum ist. Wir wollen diese Anforderung mit einer Keyword-basierten Query Methode umsetzen. Ergänze das JobRepository wie unten gezeigt.

```
package ch.zhaw.freelancer4u.repository;

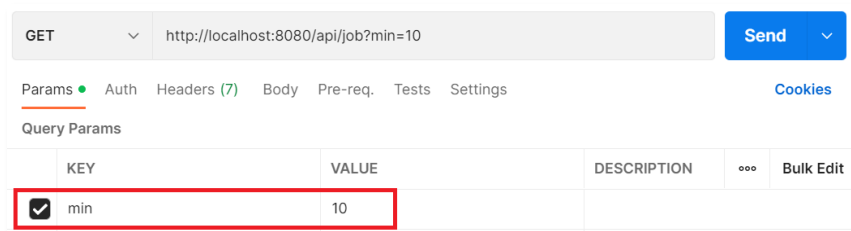
import java.util.List;
import org.springframework.data.mongodb.repository.MongoRepository;
import ch.zhaw.freelancer4u.model.Job;

public interface JobRepository extends MongoRepository<Job,String>{
    List<Job> findByEarningsGreaterThan(Double earnings);
}
```

Durch den Namen der Methode generiert das Spring Boot Framework automatisch eine passende Query. Eine Beschreibung aller unterstützten Keywords findest du in der Dokumentation unter [Repository query keywords](#). Jetzt fehlt nur noch der passende Endpoint. Ergänze den JobController wie unten gezeigt.

```
@GetMapping("/job")
public ResponseEntity<List<Job>> getAllJobs(@RequestParam(required = false) Double min) {
    List<Job> allJobs;
    if (min == null) {
        allJobs = jobRepository.findAll();
    } else {
        //...
    }
    return new ResponseEntity<>(allJobs, HttpStatus.OK);
}
```

Vervollständige die Methode und teste den Endpoint mit Postman. Variiere den Minimalwert und prüfe, ob sich die zurückgegebenen Jobs wie gewünscht filtern lassen. Erstelle zum Testen einige Jobs mit unterschiedlich hohem Lohn.



Teste auch, ob beim Weglassen des Parameters weiterhin alle Jobs zurückgegeben werden. Wenn alles funktioniert, committe deine Änderungen ins Repo.

URL	Erwartetes Ergebnis
http://localhost:8080/api/job?min=900	Alle Jobs mit Earnings > 900
http://localhost:8080/api/job?min=zehn	BAD Request (400)
http://localhost:8080/api/job	Alle erfassten Jobs

Analog zum Minimalwert soll es auch möglich sein, Jobs nach Typ gefiltert abzufragen. Die Umsetzung erfolgt ähnlich wie beim Minimalwert durch Anpassungen beim Repo und beim Controller. Die folgenden Beispiele zeigen, wie der Endpoint getestet werden kann.

URL	Erwartetes Ergebnis
http://localhost:8080/api/job?type=TEST	Alle Jobs vom Typ «Test»
http://localhost:8080/api/job?type=TOAST	BAD Request (400)

Implementiere zum Schluss noch die Variante mit beiden Filtern. **Tipp:** Mit «And» im Methodennamen können mehrere Bedingungen verknüpft werden.

URL	Erwartetes Ergebnis
http://localhost:8080/api/job?min=800&type=TEST	Alle Jobs mit Earnings > 800 vom Typ «Test»

Wenn alles funktioniert, kannst du deine Lösung committen. Verwende im Kommentar «closes» und die Nummer des Issues, um das zugehörige Issue abzuschliessen.

Gib zum Schluss die URL deines Repos erneut auf **Moodle** ab.